Университет ИТМО

Дисциплина: «Методы оптимизации»

Лабораторная работа №2

Метод ломаных (метод Пиявского)

Выполнила:

Петрова Мария К3339

Санкт-Петербург
2026

1 Цель работы

Изучить метод ломаных (метод Пиявского) и реализовать его на Python для поиска глобального минимума функции на заданном отрезке.

Также необходимо проверить работу метода на тестовой функции, построить графики и получить значение минимума, число итераций, количество вычислений функции и время работы программы.

2 Постановка задачи

Для проверки метода была выбрана одномерная функция Растригина:

$$W(x) = x^2 - 10\cos(2\pi x) + 10. \tag{1}$$

Рассматриваем функцию на отрезке

$$[-5.12; 5.12]. \tag{2}$$

Точность вычислений задаем равной

$$\varepsilon = 0.01. \tag{3}$$

Функция Растригина имеет много локальных минимумов, поэтому на ней удобно проверять работу метода глобальной оптимизации.

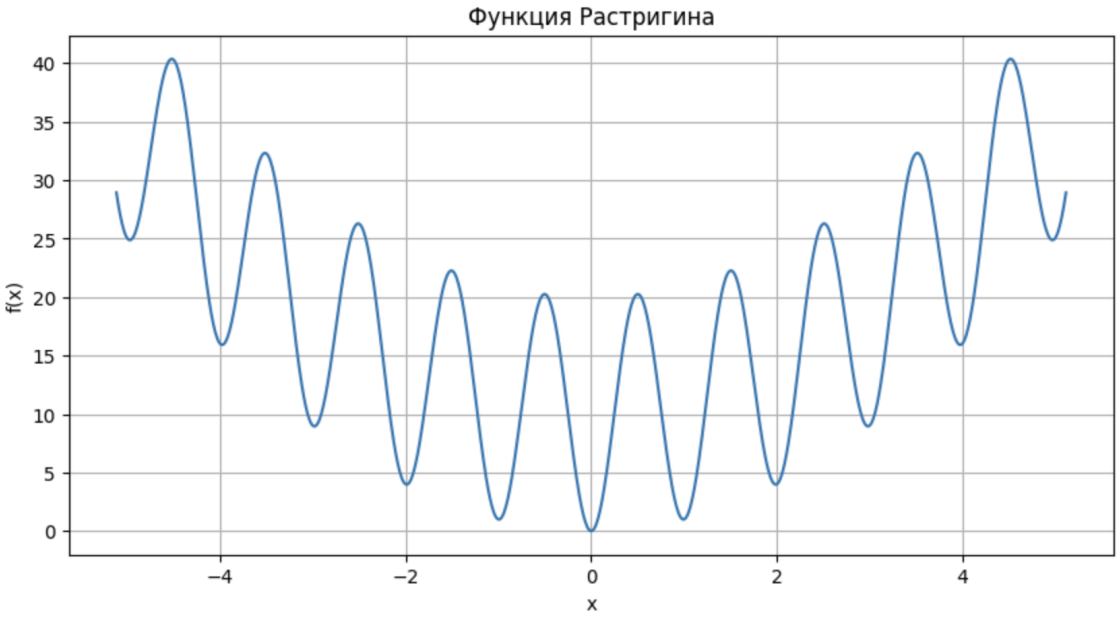


Рис. 1: График функции Растригина на отрезке $[-5.12; 5.12]$

3 Метод Пиявского

Метод Пиявского используется для поиска глобального минимума липшицевой функции на заданном отрезке.

Функция считается липшицевой, если существует константа $L > 0$, для которой выполняется условие

$$|W(u) - W(v)| \leq L|u - v|. \tag{4}$$

Основная идея метода состоит в том, что вместо прямого поиска минимума исходной функции строится специальная ломаная, которая находится ниже графика функции.

Для каждой пробной точки u_n строится вспомогательная функция

$$g_n(x, u_n) = W(u_n) - L|x - u_n|. \tag{5}$$

Из таких вспомогательных функций строится нижняя ломаная. По мере добавления новых пробных точек она уточняется и постепенно приближается к исходной функции снизу.

4 Оценка константы Липшица

Для работы метода необходимо задать константу Липшица L.

В данной реализации производная целевой функции заранее не вычисляется. Константа Липшица оценивается программно только по значениям самой функции.

На отрезке $[a, b]$ строится сетка точек. Для каждой пары соседних точек вычисляется отношение изменения функции к расстоянию между точками:

$$L_i = \frac{|W(x_{i+1}) - W(x_i)|}{|x_{i+1} - x_i|}. \tag{6}$$

Из всех полученных значений выбирается максимальное:

$$L_{\text{est}} = \max_i L_i. \tag{7}$$

Это значение используется как текущая оценка константы Липшица.

Для уточнения оценки сетка постепенно измельчается. На каждом следующем шаге число интервалов увеличивается в два раза.

Измельчение продолжается до тех пор, пока изменение оценки константы Липшица не станет достаточно малым либо не будет достигнуто максимальное количество шагов измельчения.

После этого к полученной оценке добавляется небольшой запас:

$$L = 1.2 \cdot L_{\text{est}}. \tag{8}$$

В результате было получено значение

$$L_{\text{est}} \approx 71.33, \tag{9}$$

а с учетом коэффициента запаса

$$L \approx 85.60. \tag{10}$$

Такой способ не требует заранее вычислять производную функции. Поэтому при замене целевой функции достаточно изменить только саму функцию $W(x)$.

5 Построение ломаной

Сначала рассматриваются концы отрезка:

$$u_0 = a, \qquad u_1 = b. \tag{11}$$

Из каждой пробной точки строятся две прямые с наклонами, которые задаются константой Липшица L.

Между соседними верхними вершинами вычисляется нижняя вершина ломаной. Ее координата находится по формуле

$$u_{\text{cross}} = \frac{W(u_q) - W(u_v)}{2L} + \frac{u_q + u_v}{2}. \tag{12}$$

После этого в найденной точке вычисляется значение исходной функции. Полученная точка добавляется как новая верхняя вершина, и ломаная перестраивается.

Первая ломаная строится по двум начальным точкам:

$$p_1(x) = \max\left\{g_0(x, u_0), g_1(x, u_1)\right\}. \tag{13}$$

После добавления новой точки u_2 получаем следующую ломаную:

$$p_2(x) = \max\left\{p_1(x), g_2(x, u_2)\right\}. \tag{14}$$

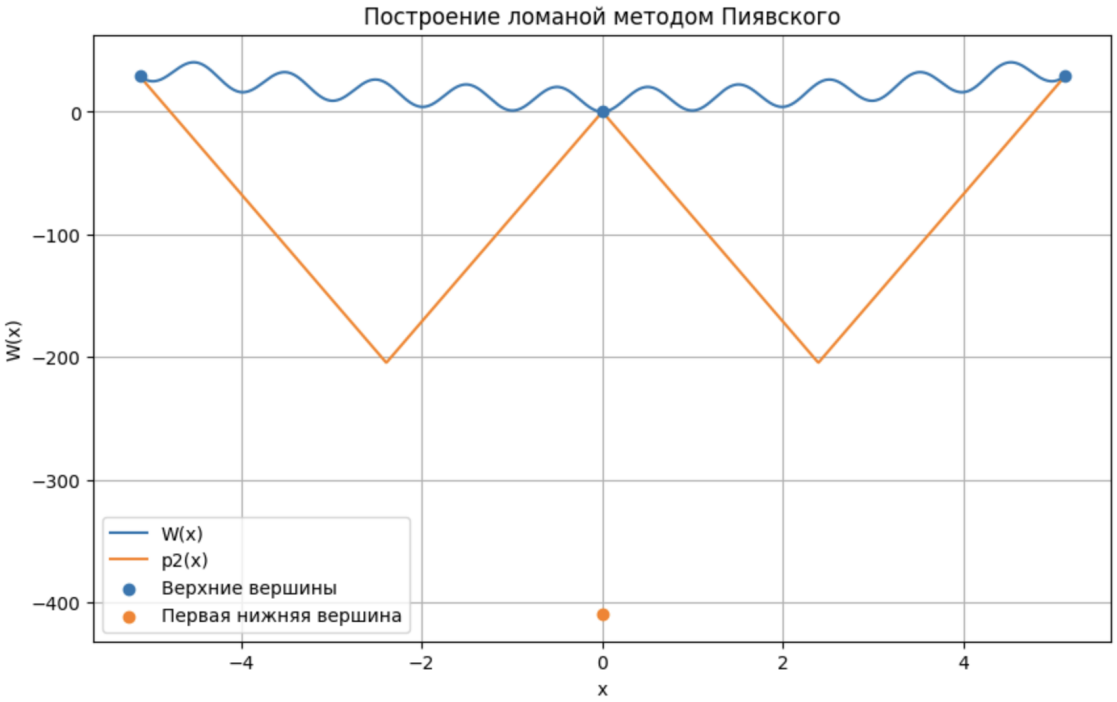


Рис. 2: Промежуточное построение ломаной

6 Алгоритм

Работа программы состоит из следующих шагов:

1. Задаются целевая функция, границы отрезка и требуемая точность.
2. Константа Липшица оценивается по значениям функции в соседних точках сетки. Для уточнения оценки сетка постепенно измельчается, после чего добавляется коэффициент запаса.
3. Вычисляются значения функции на концах отрезка.
4. Для соседних верхних вершин находятся нижние вершины ломаной.
5. Из всех нижних вершин выбирается вершина с минимальным значением.
6. Ее координата используется как новая пробная точка.
7. В новой точке вычисляется значение исходной функции.
8. Новая точка добавляется в набор пробных точек.
9. Проверяется условие остановки:

$$W(u_n) - p_{n-1}(u_n) < \varepsilon. \tag{15}$$

10. Если нужная точность еще не достигнута, шаги повторяются.

Если условие остановки выполняется, алгоритм завершается со статусом `eps_reached`.

Если заданная точность не была достигнута за максимальное число итераций, алгоритм завершается со статусом `max_iter_reached`.

После завершения алгоритма среди всех испытанных точек выбирается точка, в которой значение исходной функции минимально.

Поиск глобального минимума выполняется через построение и уточнение ломаной. Перебор значений исходной функции по мелкой сетке для поиска минимума не используется.

7 Программная реализация

Программа была написана на Python.

Для вычислений использовалась библиотека NumPy, для построения графиков — Matplotlib, а для измерения времени работы — модуль time.

Оценка константы Липшица реализована в отдельной функции `estimate_lipschitz`.

Функция получает целевую функцию и границы отрезка и самостоятельно оценивает константу Липшица по значениям функции на постепенно измельчаемой сетке.

Производная целевой функции в программе не используется.

Основной алгоритм реализован в функции `piyavsky_method`.

На каждой итерации для соседних пробных точек вычисляются нижние вершины ломаной. Затем выбирается вершина с минимальным значением, и ее координата используется как новая пробная точка.

В новой точке вычисляется значение исходной функции, после чего точка добавляется к уже проведенным испытаниям.

В программе также добавлены проверки входных данных. Проверяется корректность границ отрезка, положительность константы Липшица, требуемой точности и максимального количества итераций.

Также добавлен счетчик вычислений целевой функции.

После завершения алгоритма выбирается лучшая точка среди всех проведенных испытаний.

Код программы находится в Jupyter Notebook и размещен в репозитории.

8 Результаты

После запуска программы для функции Растригина на отрезке

$$[-5.12; 5.12] \tag{16}$$

при точности

$$\varepsilon = 0.01 \tag{17}$$

был получен следующий результат:

$$x^* = 0, \tag{18}$$

$$W(x^*) = 0. \tag{19}$$

Для функции Растригина известный глобальный минимум также находится в точке

$$x^* = 0, \qquad W(x^*) = 0. \tag{20}$$

Таким образом, найденный результат совпадает с известным глобальным минимумом функции.

Для достижения заданной точности понадобилось 200 итераций.

Значение критерия остановки составило

$$0.006597 < 0.01. \tag{21}$$

Алгоритм завершился со статусом

`eps_reached`.

Количество вычислений целевой функции непосредственно в методе Пиявского составило 202.

При оценке константы Липшица потребовалось 8135 вычислений функции.

Общее количество вычислений функции составило

$$8135 + 202 = 8337. \tag{22}$$

Время работы основного алгоритма составило примерно

$$0.014 \text{ с}. \tag{23}$$

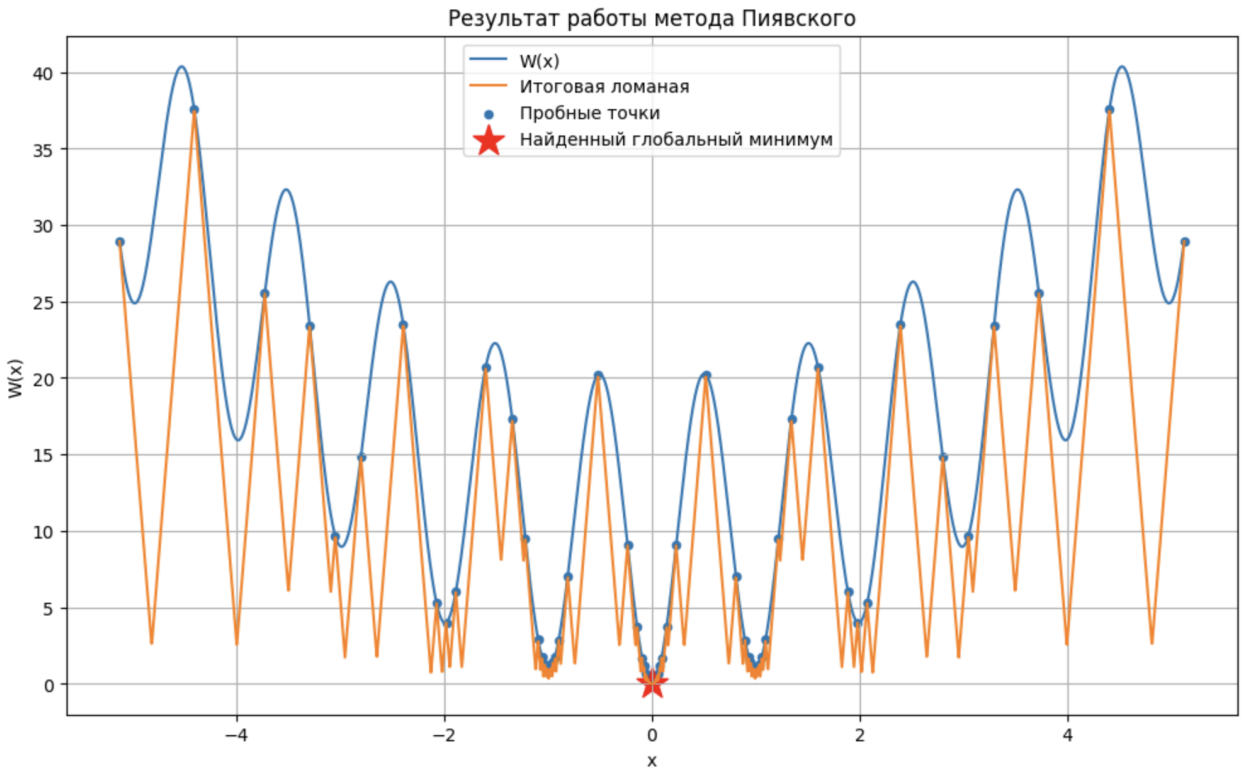


Рис. 3: Результат работы метода Пиявского

9 Вывод

В лабораторной работе был реализован метод Пиявского для поиска глобального минимума функции.

Метод был проверен на функции Растригина, у которой на выбранном отрезке имеется несколько локальных минимумов.

Константа Липшица оценивалась программно по значениям самой функции на постепенно измельчаемой сетке. Для полученной оценки использовался коэффициент запаса 1.2.

В отличие от первоначальной реализации, производную функции заранее вычислять и задавать в программе не требуется. Поэтому при замене целевой функции способ оценки константы Липшица изменять не нужно.

В основном алгоритме были добавлены проверки входных данных, статус завершения, счетчик вычислений функции и выбор лучшей точки среди всех проведенных испытаний.

В результате был найден глобальный минимум

$$x^* = 0, \qquad W(x^*) = 0. \tag{24}$$

При заданной точности $\varepsilon = 0.01$ алгоритму потребовалось 200 итераций, после чего условие остановки было выполнено.

Таким образом, метод Пиявского корректно сработал на выбранной функции и позволил найти ее глобальный минимум с заданной точностью.